

HW#0 Simulation of a HW-SW Platform



Chun-Jen Tsai
NYCU
09/05/2025

Homework Goal

- ❑ In this homework, you will learn how to simulate a HW-SW system using a waveform simulator
- ❑ You must know how to trace the execution of a program (Dhrystone) at circuit level
 - Based on your analysis using HW-SW co-simulation, you must optimize the standard library to speed up the program
 - This lab only involves software optimization, no hardware modification is necessary
- ❑ This homework is just for practice, there is no deadline

Target Technology of the Aquila SoC

- ❑ Here, we use the Aquila processor **without** the I/D-caches, the external DRAM, and the FPU
 - The RTL model of Aquila is written in Verilog
 - The model contains about 50 files, 15,000+ lines of code
 - Full-system simulation is possible with waveform simulator
 - We use Vivado simulator as the waveform simulator in this course

- ❑ Toolchains used for HW-SW system development:
 - HW: Vivado 2025.1 for FPGA circuit design & simulation
 - SW: RISC-V 32-bit GCC 15.1.0 toolchain

Introduction to AMD/Xilinx EDA Tools

- ❑ Vivado (for HW design only)
 - Support traditional HW design flow using Verilog or VHDL
 - Includes circuit synthesizer, simulator, and embedded logic analyzer

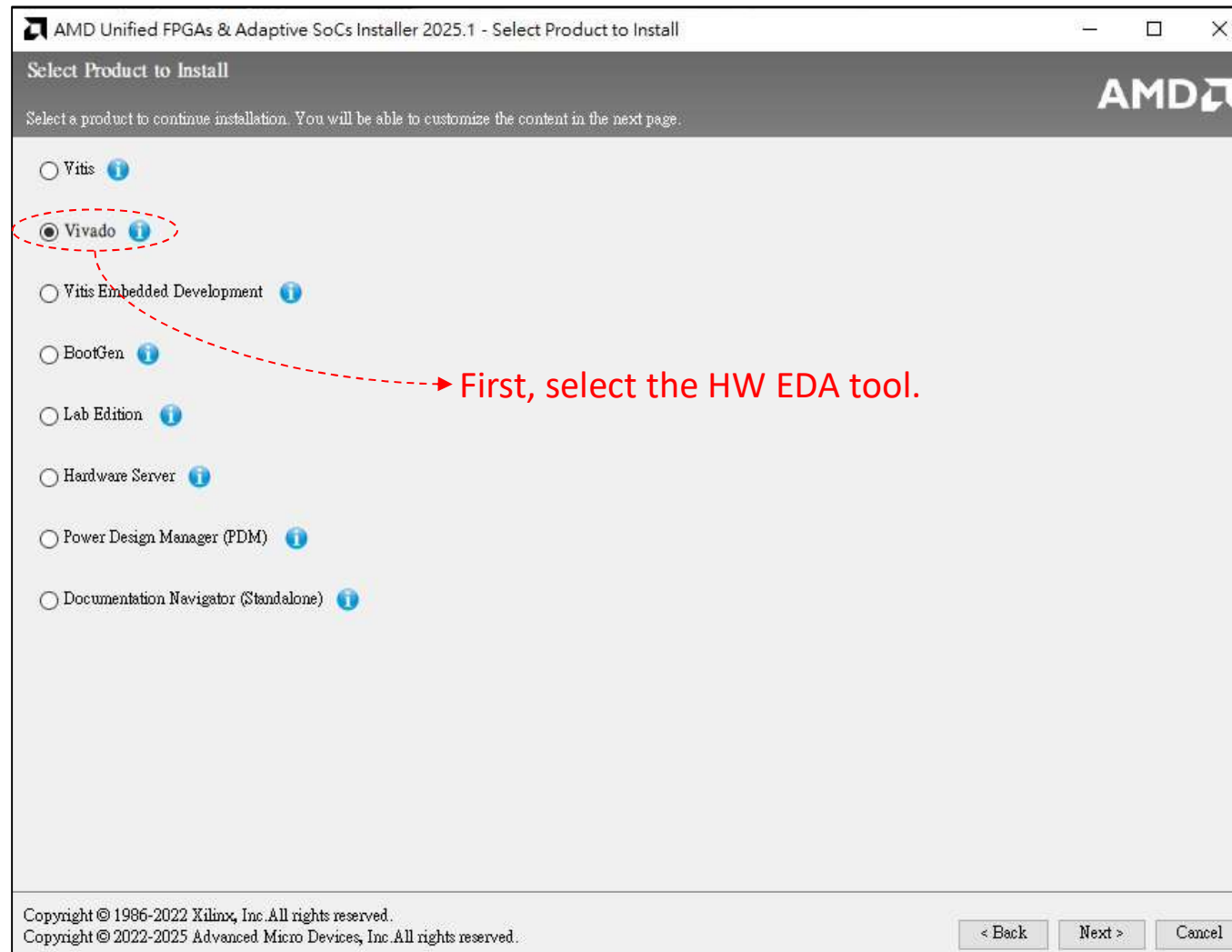
- ❑ Vitis (for both HW/SW design)
 - Software IDE (only for ARM and Microblaze processors)
 - High-Level Synthesis (HLS) HW design using C/C++
 - Support Xilinx FPGA & AI chips
 - Rely on Vivado for FPGA HW implementation

- ❑ *As of 2024, Xilinx has merged Vivado into the Vitis package. So, you have to use the Vitis installer to install Vivado*

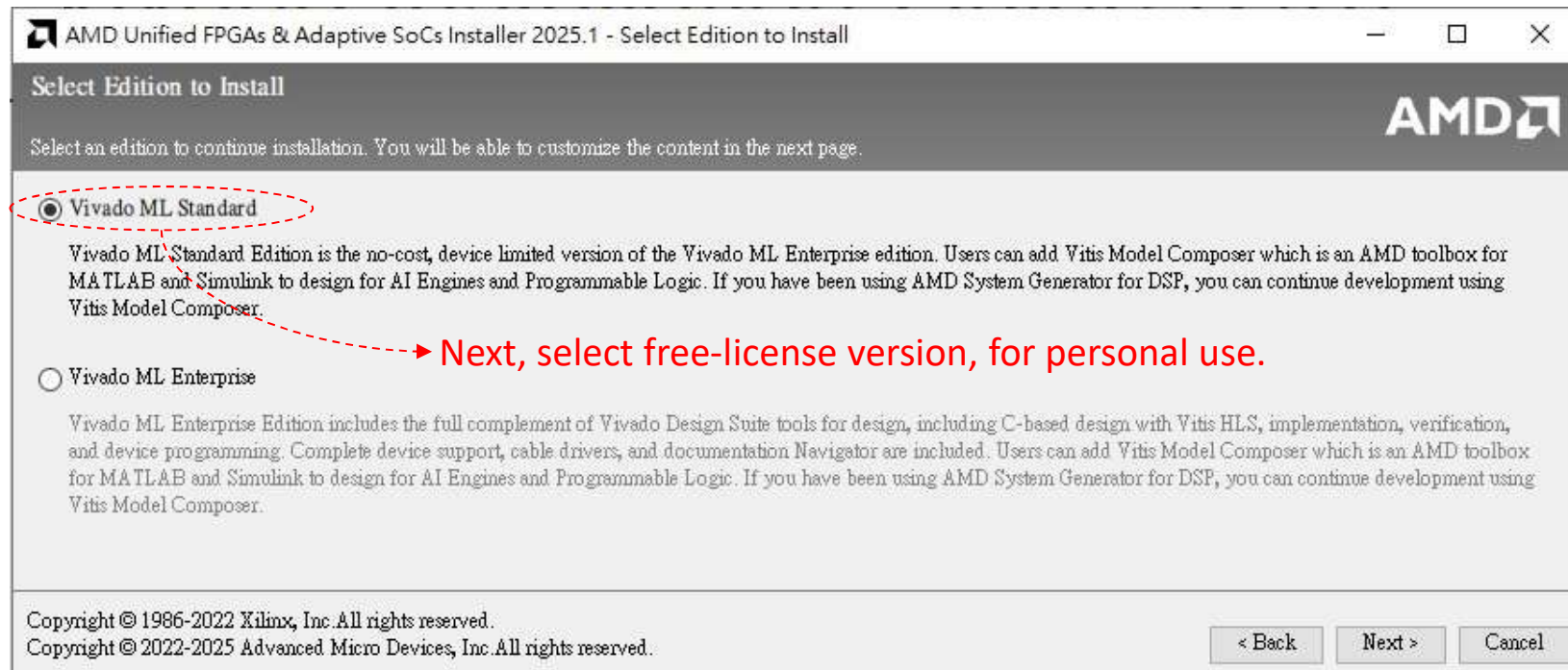
Installation of Vivado Design Suite

- ❑ You can download the installer for Windows or Linux:
 - <https://www.xilinx.com/support/download/index.html/content/xilinx/en/downloadNav/vitis.html>
 - Download & install “AMD Vitis™ Unified Software Platform 2025.1”
 - The installer includes “Vivado ML 2025.1”
 - MacOS is not supported by AMD/Xilinx!
- ❑ Full installation requires 100+ GiB of disk space, based on the number of FPGA devices you have selected
 - You must register a free Xilinx account to get a license to use the EDA tool

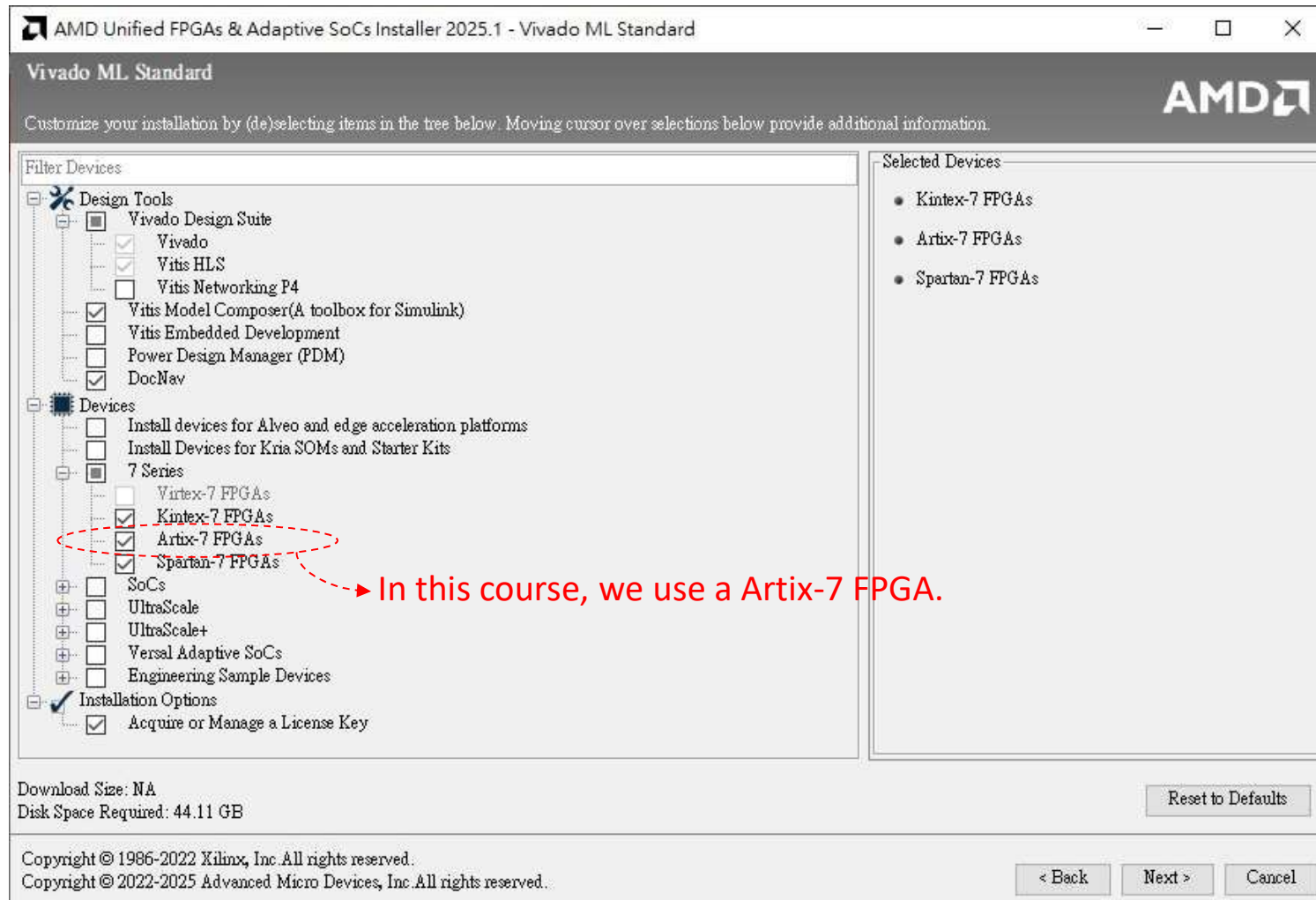
Vivado Installation Guide (1/3)



Vivado Installation Guide (2/3)



Vivado Installation Guide (3/3)



Creation of an Aquila SoC Workspace

- ❑ Download and unzip `aquila_build.zip` from E3
 - It contains a TCL script that generates the Aquila workspace

```
aquila_build/ +- src/ -----> Source code of the Aquila SoC
               +- tcl/
                 +- build.tcl -----> Script to create Aquila without FPU
                 +- build_fpu.tcl --> Script to create Aquila with FPU
```

- ❑ In the directory, type the following command under a Windows command prompt:

```
> C:\<INST_DIR>\2025.1\Vivado\bin\vivado.bat -mode batch -source tcl\build.tcl
```

Or, if you use Linux, under a bash prompt:

```
$ <INST_DIR>/2025.1/Vivado/bin/vivado -mode batch -source tcl\build.tcl
```

The Aquila workspace will be in `./aquila_ap/`,
and the Vivado GUI will be launched automatically

Open the Workspace Manually

- ❑ If you want to manually launch the Vivado GUI to open the workspace, follow the instructions:
 - For Windows, double-click the file `aquila_ap.xpr` in the `./aquila_ap/` directory
 - Under Linux bash, you can type the command:

```
$ <INST_DIR>/2025.1/Vivado/bin/vivado aquila_mpd.xpr
```

Overview of the Aquila Workspace

The screenshot displays the Vivado 2025.1 IDE interface for the Aquila workspace. The top menu bar includes File, Edit, Flow, Tools, Reports, Window, Layout, View, and Help. The toolbar contains various icons for file operations and simulation. The left sidebar shows the Flow Navigator with sections for PROJECT MANAGER, IP INTEGRATOR, SIMULATION, RTL ANALYSIS, SYNTHESIS, IMPLEMENTATION, and PROGRAM AND DEBUG. The central Project Manager window shows the Sources list with Design Sources (35) and Verilog Header (1). The 'soc_top (soc_top.v) (5)' module is highlighted with a red dashed box. The right sidebar shows the Project Summary window with the Overview tab selected. The Settings section is highlighted with a red dashed box and a red arrow pointing to the text 'Configurable parameters of Aquila SoC'. The Top module name is also highlighted with a red dashed box and a red arrow pointing to the text 'Top-level synthesizable module.'.

Project Summary Overview:

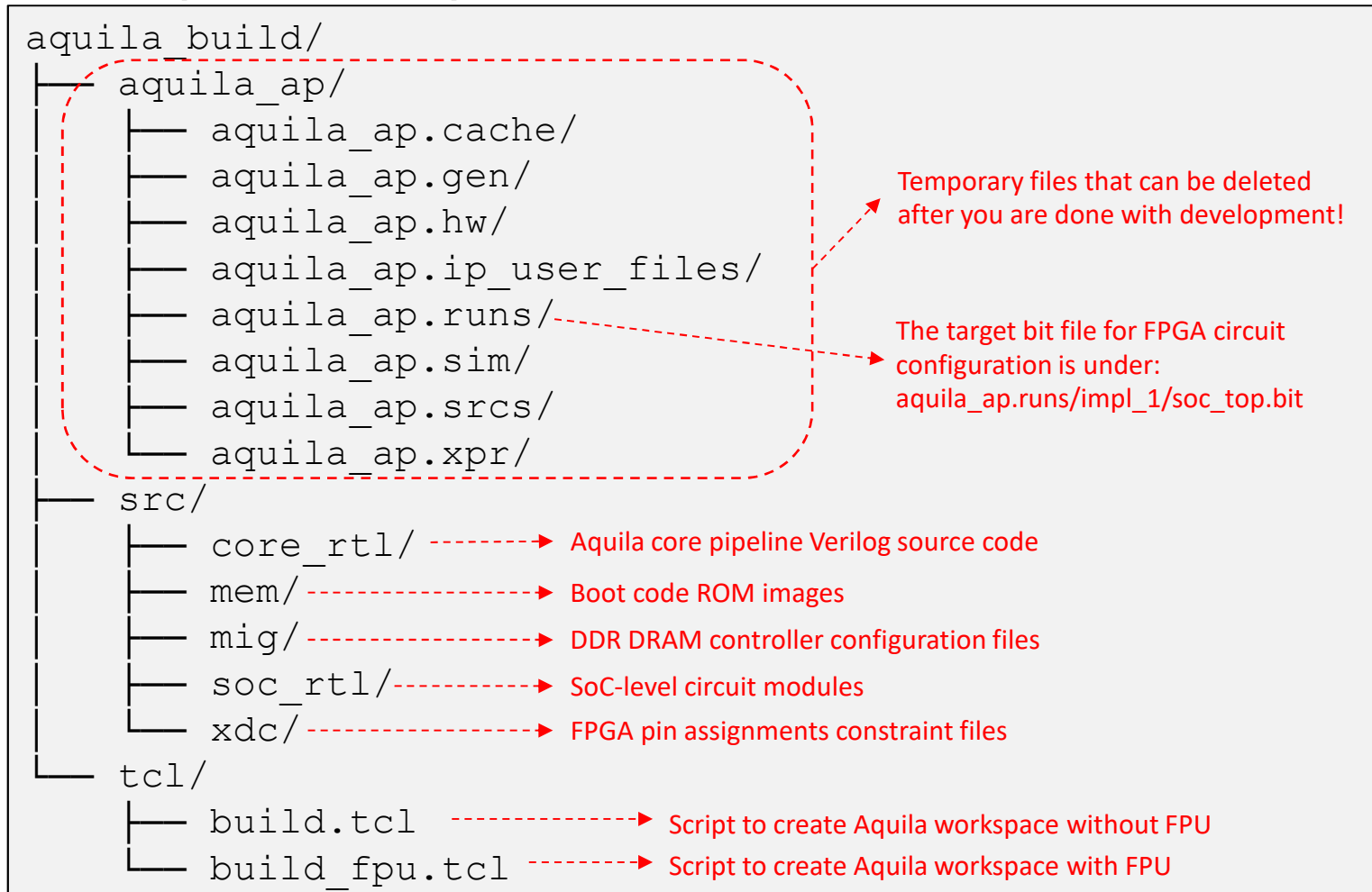
- Project name: aquila_ap
- Project location: D:/xilinx/aquila/aquila_fp/aquila_ap
- Product family: Artix-7
- Project part: xc7a100tcsa324-1
- Top module name: soc_top
- Target language: Verilog
- Simulator language: Mixed
- Target Simulator: Vivado Simulator

Design Runs Table:

Name	Constrai...	Status	W...	Methodol...	B...	U...	S...	Elap...	TP...	Total Po...	Failed Ro...	Run Strategy
synth_1	constrs_1	Not started										Vivado Synthesis Default
impl_1	constrs_1	Not started										Vivado Implementation D

Directory Structure of the Workspace

❑ The Aquila workspace structure:



Configuration of Aquila SoC

- ❑ The Verilog header file, `aquila_config.h`, contains some configurable features of the processor

```
                . . . . .
// Tightly-Coupled Memory Size
`define TCM_SIZE_IN_WORDS 16384    // 64KB

// DDRx memory is only accessible via cache controller
//`define ENABLE_DDRx_MEMORY
`define ICACHE_SIZE 8    // Instruction cache size in KB
`define DCACHE_SIZE 8    // Data cache size in KB

// Branch Prediction
`define ENABLE_BRANCH_PREDICTION
                . . . . .
```

Installation of the SW Toolchain

- ❑ You can build the latest GCC under Linux (or WSL[†]):

- First, clone the latest source of the compiler:

```
$ git clone https://github.com/riscv-collab/riscv-gnu-toolchain.git
```

- Build the Newlib version with the configuration parameters:

```
$ cd riscv-gnu-toolchain/  
$ ./configure --prefix=/opt/riscv --with-arch=rv32imad_zicsr_zifencei --with-abi=ilp32d --enable-multilib  
$ sudo make -j16
```

- After installation, you should set the shell variables:

```
$ export PATH=$PATH:/opt/riscv/bin  
$ export RISCVC=/opt/riscv
```

[†] If you use Windows, you can install WSL to use the toolchain.

Sample Software

❑ The sample software source tree of Aquila

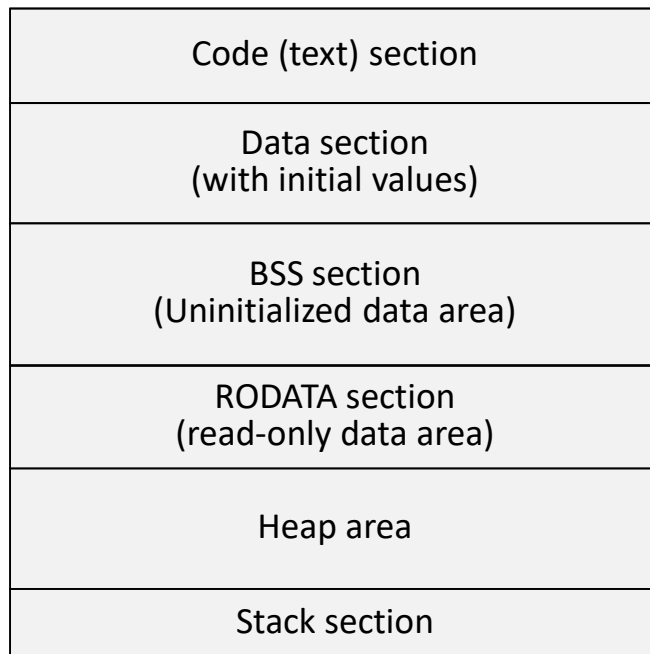
```
aquila_sw +- uartboot/  → boot code that loads an ELF file
           +- elibc/     → A small "unoptimized" C library
           +- CoreMark/  → The CoreMark benchmark program
           +- Dhrystone/ → The Dhrystone benchmark program
           +- pi/        → Computing 5,000 digits of pi
           +- rtos_mmul/ → A multi-thread program of FreeRTOS
           +- cnn_ocr2/  → A hand-written digits recognition CNN
```

❑ Download `aquila_sw.tgz` from E3. Unpack the file under your Linux system.

- You can build a program by typing “make” in each directory.
- Please read and understand the Makefile!

Runtime Memory MAP

- ❑ A typical runtime memory map:



These sections do not have to occupy contiguous memory areas.

The POSIX `*.elf` executable file format allows a non-contiguous memory layout.

The system program, **loader**, will parse and load the program into the correct memory areas.

- ❑ A linker script (`*.ld`) can be used to control the memory layout of an executable file

Linker Script Example

- ❑ Default linker script for Dhrystone:

```
__stack_size = 0x800;
__heap_size  = 0x2000;

MEMORY
{
    code_ram    (rx!rw) : ORIGIN = 0x01000, LENGTH = 0x7000
    data_ram    (rw!x)  : ORIGIN = 0x08000, LENGTH = 0x8000
}

ENTRY(crt0)

SECTIONS
{
    .text :
    {
        *(.text*)
    } > code_ram

    .data :
    {
        *(.data)
        *(.bss)
        *(.rodata*)
    } > data_ram

    .heap : ALIGN(0x10)
    {
        __heap_start = .;
        . += __heap_size;
    } > data_ram

    .stack : ALIGN(0x10)
    {
        . += __stack_size;
        __stack_top = .;
    } > data_ram
}
```

The compiler will read the linker script and generate machine code accordingly.

Program Binary File Formats

- ❑ We use two different program binary file formats:
 - `*.mem` – used to initialize the on-chip memory (i.e. boot code)
 - `*.elf` – the standard UNIX Executable and Linkable Format

- ❑ To load and run an ELF file, the on-chip memory must be initialized with the `uartboot.mem`.
 - `uartboot` will do the following things
 - Print an prompt in the terminal window
 - Asks you to transfer an `*.elf` program from the host PC through the UART to the Arty FPGA board for execution
 - Put the ELF file sections into the proper memory areas

Simulation Using Vivado Simulator

- ❑ In Aquila workspace, there are two top-level modules:
 - `soc_top.v` – top module for circuit synthesis
 - `soc_tb.v` – testbench for circuit simulation

- ❑ For simulation, the UART controller, `uart.v`, reads an ELF file from the host PC disk drive and “simulates” the transfer of the file to the UART input port
 - The ELF file path/name is defined in `aquila_config.h`

Run Behavioral Simulation

The screenshot displays the Vivado 2025.1 IDE interface for a project named 'aquila_ap'. The 'Flow Navigator' on the left shows the project hierarchy with 'SIMULATION' selected. A red dashed arrow points to the 'Run Simulation' button under the 'SIMULATION' section, with a text box stating 'Click this to run simulation.'.

The 'PROJECT MANAGER - aquila_ap' window shows the 'Sources' tab with 'aquila_config.vh' selected. The 'Hierarchy' tab is also visible.

The 'Project Summary' window shows the 'aquila_config.vh' file. A red dashed arrow points to the line defining the simulation program path, with a text box stating 'ELF program to be executed!'. The code snippet is as follows:

```
162 // Branch Prediction
163 `define ENABLE_BRANCH_PREDICTION
164
165 // Atomic Unit
166 `define ENABLE_ATOMIC_UNIT
167
168 // Muldiv Integer multiplier
169 `define ENABLE_FAST_MULTIPLY
170
171 // Fetch
172 `define NOP 32'h00000013
173
174 // SIM_FNAME defines the RISC-V program path of an ELF file for simulation.
175 `define SIM_FNAME "D://dhry.elf"
176
```

The 'Design Runs' window at the bottom shows a table with columns: Name, Constrai..., Status, W..., Methodol..., B..., U..., S..., Elap..., TP..., Total Po..., Failed Ro..., and Run Strategy. The table contains two rows: 'synth_1' and 'impl_1', both with a status of 'Not started'.

Name	Constrai...	Status	W...	Methodol...	B...	U...	S...	Elap...	TP...	Total Po...	Failed Ro...	Run Strategy
synth_1	constrs_1	Not started										Vivado Synthesi
impl_1	constrs_1	Not started										Vivado Impleme

Vivado Simulator Window

The screenshot displays the Vivado Simulator Window for a project named 'aquila_mpd'. The interface includes a menu bar, a toolbar, and several panels. Red dashed boxes and arrows highlight key features:

- Simulation time:** A red dashed box highlights the '35 ms' value in the simulation time display.
- Zoom waveform properly to fit window:** A red dashed box highlights the zoom controls in the waveform viewer.
- Pick the module whose signals you want to observe!** A red dashed box highlights the 'soc_tb' module in the Scope panel.
- Watch TCL console window for simulated output!** A red dashed box highlights the TCL Console window, which shows the output: 'Copyright (c) 2019-2024, EISL@NYCU, Hsinchu, Taiwan. The Aquila SoC is ready. Waiting for an ELF file to be sent from the UART ...'.

The Scope panel shows the following hierarchy:

- soc_tb
 - Aquila_SoC
 - uart
 - gbl

The Ob panel shows the following signals:

- sys_reset
- sys_clock
- usr_reset
- ui_clk
- ui_rst
- clk
- rst
- uart_rx
- uart_tx
- IMEM_strobe

The waveform viewer shows a table of signals and their values:

Name	Value
clk	0
rst	0
uart_rx	1
uart_tx	1
IMEM_strobe	Z
IMEM_a[31:0]	ZZZZZZZZ
IMEM_done	Z

On Simulation of `printf()`

- ❑ Note that the testbench and `uart.v` also support the simulation of output from the C library `printf()`
 - At circuit level, `printf()` sends ASCIIs to the `uart` module
 - In simulation mode, the `uart` module will send the ASCIIs to the “Tcl Console” of Vivado

- ❑ There is a trap in `uart.v` such that when the ASCII code `0x03` is printed, the simulation will terminate
 - The boot code, `uartboot.c`, of Aquila will print ‘0x03’ when the execution of a program is done.

Simulated Results

The screenshot displays the Vivado 2025.1 interface during a behavioral simulation. The top menu bar includes File, Edit, Flow, Tools, Reports, Window, Layout, View, Run, and Help. The toolbar shows various simulation and project management icons. The left sidebar contains the Project Manager, IP Integrator, and Simulation sections. The main workspace is divided into several panes:

- Scope:** A table listing the design hierarchy.

Name	Design Unit
soc_tb	soc_tb
Aquila_SoC	aquila_top
UART	uart
Memory_Arbitrer	mem_arb
MIG	memctrl_1
gbl	gbl
- Values:** A table showing the current values of signals.

Name	Value
sys_reset	0
sys_clock	1
usr_reset	0
ui_clk	1
ui_rst	0
clk	1
rst	0
- Code Editor:** Displays the Verilog code for `soc_tb.v`. The code includes a testbench for the `Aquila_SoC` module, with a `$finish()` statement highlighted in yellow.

```
//  
246 always @(posedge clk)  
247 begin  
248   if (simulation_finished) begin  
249     $display();  
250     $display("Simulation finished.");  
251     $finish();  
252   end  
253 end  
254
```
- Tcl Console:** Shows the output of the simulation. A red dashed box highlights the output messages used to verify modifications to the C library. Another red dashed box highlights the DMIPS/MHz value.

```
should be: 1  
Str_1_Loc: DHRYSTONE PROGRAM, 1'ST STRING  
should be: DHRYSTONE PROGRAM, 1'ST STRING  
Str_2_Loc: DHRYSTONE PROGRAM, 2'ND STRING  
should be: DHRYSTONE PROGRAM, 2'ND STRING  
  
It tooks 0.01 seconds.  
Microseconds for one run through Dhrystone: 14.806000  
Dhrystones per Second: 67540.2  
VAX MIPS: 38.4  
DMIPS/Mhz: 0.77
```

Output messages used to verify that your modifications to the C library is correct!

DMIPS/MHz, when dhry.elf is compiled using gcc 15.1.0

Tracing the Execution of a Function

- ❑ One of the reasons that the DMIPS of Aquila is low is because that the C library (`ellibc`) is not optimized!
 - On the other hand, the popular Newlib C library for embedded system is optimized
- ❑ For example, you can trace the execution of the `strcpy()` at circuit level, and analyze why the processor cannot execute the function efficiently
 - The `dhry.map` tells you that the start address of the function
 - `strcpy()` begins at `0x00002e6c` in my build (gcc 15.1.0),
 - In the simulator, search for the code address `0x2e6c`, and start tracing the waveform
 - pay attention to the **stall cycles** for load/store instruction execution

Cross-Referencing the Assembly

- ❑ After making the `dhrv.elf` program, there is a text file, `dhrv.objdump`, that contains the assembly code of the program
 - This file is helpful for you to analyze the waveform
- ❑ To understand the assembly code, you need to know:
 - The instruction set architecture (ISA)
 - The Application Binary Interface (ABI) of the CPU

Sample Assembly Code

❑ The assembly code of `dhry.objdump` :

Aquila execution
begins here!



```
dhry.elf:      file format elf32-littleriscv

Disassembly of section .text:

00001a5c <crt0>:
1a5c:      ff010113      addi      sp,sp,-16
1a60:      00112623      swra,12(sp)
1a64:      0000b2b7      lui      t0,0xb
1a68:      1422aa23      swsp,340(t0) # b154 <sp_store>
1a6c:      0000b2b7      lui      t0,0xb
1a70:      0f82a103      lwsp,248(t0) # b0f8 <stack_top>
1a74:      fb8ff0ef      jal      122c <main>
1a78:      0000b2b7      lui      t0,0xb
1a7c:      1542a103      lwsp,340(t0) # b154 <sp_store>
1a80:      00c12083      lwra,12(sp)
1a84:      01010113      addi      sp,sp,16
1a88:      00008067      ret

00001a8c <getchar>:
1a8c:      ff010113      addi      sp,sp,-16
1a90:      00112623      swra,12(sp)
```

Dhrystone Benchmarks Issues

- ❑ There is no perfect benchmarks. For Dhrystone, it's much less than perfect[†]:
 - Too many fixed-length string operations (`strcpy()` and `strcmp()`)
 - Code/data size too small to test cache performance
 - Dirty compilers that optimize for Dhrystone can achieve extra 50% higher DIMPS numbers
 - Did not take into account architecture features (e.g., RISC, VLIW, SIMD, and superscalar)
 - Code patterns do not reflect modern applications
(Big Question: is CPU critical for modern applications?)

[†] https://www.eembc.org/techlit/datasheets/dhrystone_wp.pdf

Your Homework

- ❑ Get familiar with the behavior simulation of a complex HW-SW system

- ❑ Rewrite `strcpy()` and `strcmp()`, see if you can increase the DMIPS/MHz performance
 - A useful reference is the *Bit Twiddling Hacks*[†] from Stanford
 - You can also study other optimized standard C libraries, such as Newlib, to see how others do it
 - Don't forget to use the simulator to analyze and compare the execution of your optimized code against the original code

[†] <https://graphics.stanford.edu/~seander/bithacks.html>